

Decide Early or Sum First: Where Analog Computing Works, and Why

Sep 29, 2026 · @wofl

Summary

Analog hardware is excellent at one big multiply-and-sum and poor at long chains of decisions. The difference is restoration: whether each step snaps its signal back to a clean value before noise can pile up.

- **Logic needs restoration.** With noise of a tenth of the signal swing on every wire, a bit passed down a chain of NOT gates without restoring gain is a coin toss after 128 gates. Steep, restoring gates carry it through 512 gates with no errors in 2,000 trials.
- **One analog multiply works.** A 256-input, 16-output multiply done in one shot of physics returns about 6 effective bits: 6.2 for an electronic crossbar and 5.8 for optics, read by an 8-bit converter.
- **Each medium pays for precision in its own currency.** Where that noise dominates, the crossbar gains a bit each time its weight-programming error halves, and optics gains a bit for every fourfold increase in photons.
- **A wider converter can't add precision the physics didn't deliver.** Moving the crossbar from an 8-bit to a 12-bit converter lifts it only from 6.2 to 6.3 effective bits.
- **Long analog sums stop being exact.** Summing ± 1 values, the crossbar is exactly right 99.34% of the time at 256 terms, but only 49.49% at 4,096.

One rule covers both halves: **decide early and often, and noise never adds up; sum a lot before deciding, and it does.** Every number here comes from a deterministic simulator, shipped with this paper, and can be reproduced with the commands in Appendix B. The noise settings are illustrative, not measurements of any chip.

Why ask this now

Analog computing is back because AI's arithmetic is expensive, and physics can do some of it almost for free. IBM Research has built an analog AI chip with 35 million phase-change memory devices across 34 tiles, doing neural-network multiplies inside the memory itself ([Ambrogio et al., 2023](#)). A Cornell group has classified handwritten digits optically at about 3 detected photons per multiply ([Wang et al., 2022](#)).

The recurring question is where the idea holds up and where it falls over. This paper answers it with one mechanism, restoration, tested in two settings: chains of logic gates, and single analog matrix multiplies, electronic and optical.

It replaces an earlier draft (2024) that proposed building reasoning hardware from analog logic gates; the evidence below is why that direction was dropped. The simulator behind it is behavioural. It shows each effect's shape and cause, not the numbers of any real chip.

The one idea: restoration

A noisy signal survives a long journey only if every stage snaps it back to a clean value before passing it on. Physics will do sums for free, but it never checks its work; restoration is the checking.

Think of the whisper game. If each player repeats exactly what they think they heard, garble and all, mistakes pile up, and a long enough row ends in nonsense. If each player snaps what they heard to the nearest real word, small garbles vanish at every step, and only a big mishearing gets through.

In electronics the snapping is gain. A gate's output stage swings hard towards 0 or 1 near its switching point, so small wobbles near the rails get squashed flat. Only a disturbance big enough to cross the midpoint flips the result; the distance to that midpoint is the noise margin.

Restoration has one precondition that matters later: the right answers must sit on a grid, with room between them. Logic's answers always do, since every signal is meant to be exactly 0 or exactly 1.

How it was tested

Every result comes from analog-nand, a Rust simulator shipped with this paper, with no dependencies and fixed random seeds, so each number reproduces exactly.

The logic model. Each gate takes four steps:

1. It reads its noisy input levels, clamped to the 0-to-1 supply range.
2. It computes the exact soft version of its truth table: the chance its output is 1 if each input were independently 1 with the given probability. For NAND this is $1 - ab$.
3. A restoring output stage pushes the result towards 0 or 1. Its steepness is the gain knob, where 0 means no restoration.
4. The output wire adds Gaussian noise, and the next gate reads that noisy level.

An observer reads each final output as 1 above 0.5. The score is the share of output bits read wrong, against a perfect digital run on the same inputs.

The multiply model. One matrix-vector multiply, $y = Wx$, happens in a single shot, for two media on equal terms:

- **Electronic crossbar.** Weights are conductances and inputs are voltages; currents add up on a shared wire. Noise: 2% weight-programming error, 1% read flicker per cell, and sense-amplifier noise of 0.1% of full scale.
- **Optics with balanced detection.** Inputs are light intensities and weights are modulator settings; positive and negative terms land on two photodetectors. Noise: 1% weight-setting error, 1% laser flicker, detector noise of 0.1% of full scale, and shot noise from counting whole photons (1,000 per full-scale multiply, drawn from a true Poisson distribution).

An ideal converter then reads each output across ± 5 standard deviations of the answer. The score is effective bits: how many bits a perfect converter would need to match the answer's accuracy.

Both media see exactly the same random tasks, so every comparison is paired. The digital circuits are checked against ordinary arithmetic on every possible input, and 38 automated tests lock the laws reported below.

Result 1: logic needs restoration

Without restoration, a bit passed down a chain of NOT gates is a coin toss after 128 gates; with steep gates it survives 512 untouched (chart below).

The same pattern holds in a real circuit. On an 8-bit adder built from 72 NAND gates, errors show without restoration at noise 0.02 and reach 0.9% of output bits at 0.04. With gain 8 the adder is still error-free at noise 0.08, then climbs fast: 0.03% at 0.10 and 15% at 0.20.

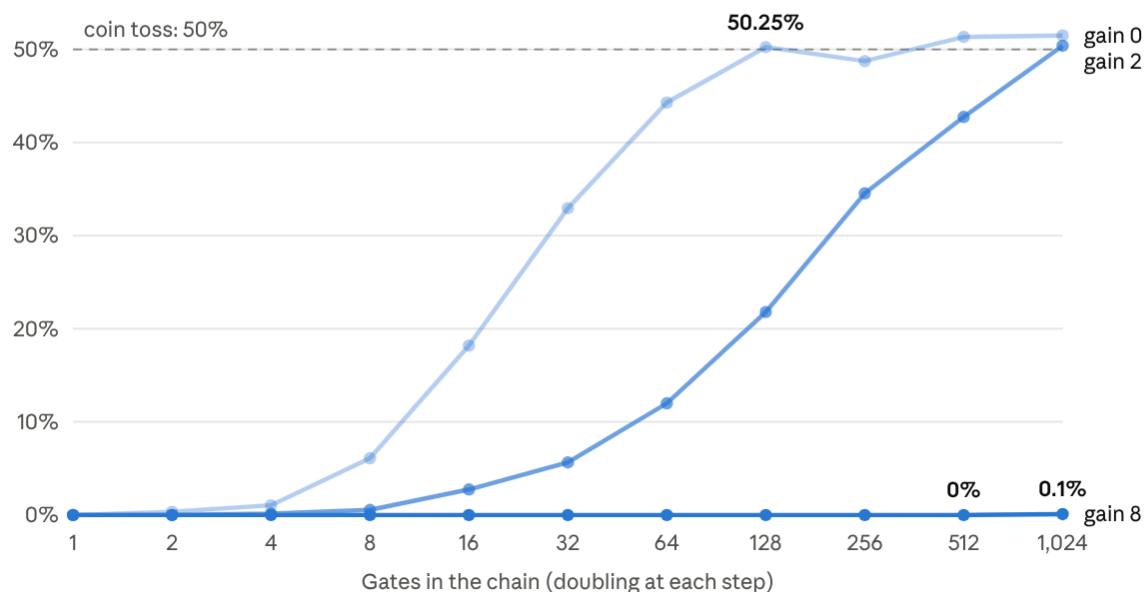
That climb is the noise margin running out. Past it, one noisy wire can flip a gate outright, and restoration cannot undo a wrong decision; it passes it on, cleanly.

One run shows the mechanism. The same full adder was run twice with identical noise, at gain 0 and at gain 8, and the same +0.27 kick of noise hit one internal wire both times.

At gain 0, wobble from earlier gates had already dragged that wire to 0.46, just short of the 0.5 midpoint. The kick pushed it over, and the sum came out wrong. At gain 8 every earlier gate had re-snapped, so the wire started at 0.004 and the same kick left it at 0.27: still a clean 0.

No restoration: a coin toss by 128 gates. Steep gates: clean through 512

Bits arriving wrong after a chain of NOT gates; noise 0.1 on every wire; 2,000 trials per point



analog-nand depth · noise 0.1, gains 0, 2 and 8, 2,000 trials per point, seed 42 · run 2026-09-29

Gain 2 restores a little, which only slows the drift. Gain 8's rare errors at 1,024 gates come from single gates flipping outright, so they grow with the length of the chain instead of piling up.

Result 2: one analog multiply works

One shot of physics does 4,096 multiplies and 16 sums at once and returns about 6 effective bits: 6.2 for the crossbar, 5.8 for optics. Many neural networks run at 4 to 8 bits, so this is the job analog does well.

Each noise source on its own (read by the widest converter, so the converter hides nothing), then everything together through an 8-bit converter:

Medium	Noise source	Effective bits
Electronic crossbar	Weight-programming error, 2%	6.4
Electronic crossbar	Read flicker, 1% per cell	8.2
Electronic crossbar	Sense-amplifier noise, 0.1% of full scale	9.2
Electronic crossbar	Everything, through an 8-bit converter	6.2
Optics	Weight-setting error, 1%	7.4
Optics	Shot noise, 1,000 photons per multiply	5.9

Medium	Noise source	Effective bits
Optics	Laser flicker, 1%	8.2
Optics	Detector noise, 0.1% of full scale	9.2
Optics	Everything, through an 8-bit converter	5.8

The weakest source sets the result: weight programming for the crossbar, photon counting for optics.

Each medium pays for precision in its own currency. Halving the crossbar's programming error from 2% to 1% lifts it from 6.2 to 6.9 bits. Ten times the light, from 1,000 to 10,000 photons per multiply, lifts optics from 5.8 to 6.7.

In isolation both laws are exact: each halving of weight error buys one bit, and each fourfold increase in photons buys one bit, because shot noise grows only as the square root of the photon count. Automated tests lock both laws.

A wider converter can't add precision the physics didn't deliver. With perfect physics, effective bits equal converter bits exactly: 4.0, 6.0, 8.0, 10.0 and 12.0. With the crossbar's noise, a 12-bit converter returns 6.3 bits, barely above the 8-bit converter's 6.2.

A common rule of thumb is that each extra converter bit doubles its energy, so the converter should match the physics, not exceed it.

Result 3: the catch

Long analog sums stop being exact: summing ± 1 values, the crossbar is exactly right 99.34% of the time at 256 terms, but only 49.49% at 4,096 (chart below).

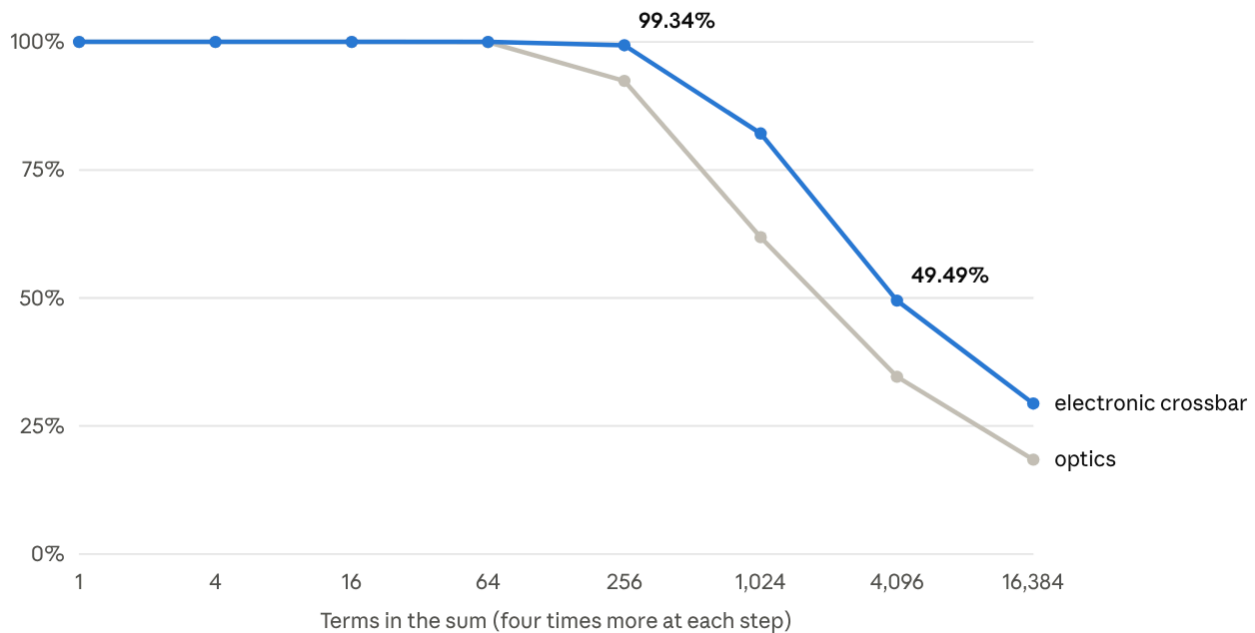
With one input, an analog multiply is as exact as a logic gate: right answers sit a whole step apart, and the noise is tiny next to that gap. Every term added to the sum brings its own noise, but the gap between right answers never grows.

So the chance of an exact answer falls as the sum grows, and the converter needs more bits just to tell the answers apart: 7 bits at 256 terms, 10 at 16,384. Optics loses exactness sooner (92.39% at 256 terms), because counting photons adds noise to every term.

Here the converter is ideal and just wide enough, so every miss is the physics' fault. The simulation matches a pen-and-paper Gaussian estimate to within a hundredth of a percent: 99.35% predicted at 256 terms, 99.34% measured (Appendix A).

Long analog sums stop being exact: 99.34% right at 256 terms, 49.49% at 4,096

Answers exactly right when summing ± 1 values with an ideal converter; 16 outputs per shot



analog-nand exact · 1-bit values, 16 outputs per shot, seed 7 · run 2026-09-29

Both media stay exact up to 64 terms; optics falls away sooner because every photon count adds its own noise.

One rule for both halves

Decide early and often, and noise never adds up; sum a lot before deciding, and it does.

- **Logic decides after every 1 to 3 inputs, with a whole unit of margin each time.** That is why restoring gates carry a bit through 512 stages untouched.
- **An analog multiplier sums thousands of terms before anything decides.** That makes it efficient and unavoidably approximate: well suited to neural networks, which tolerate a few bits of error, and the wrong tool for exact arithmetic or logic.
- **A converter restores a signal only when the right answers sit on its grid with room to spare.** Logic's answers always do; a long analog sum's answers do not, which is what Result 3 measures.

Three design habits follow. Keep analog steps wide but shallow: one big sum, then a digital decision. Size the converter to the physics' noise floor, not beyond it, and spend on whichever currency is cheaper: device accuracy for electronics, light for optics.

Neither medium wins on principle. In this model both reach about 6 bits by different routes: optics pays in photons, which is energy, and the crossbar pays in device accuracy, which is fabrication and calibration. Which is cheaper in practice depends on real device figures, which this model deliberately leaves as knobs.

What this paper does not show

The simulator is behavioural: it shows the shape of each effect and its cause, not the numbers of any real chip.

- **The noise settings are illustrative.** They are knobs chosen to be plausible; comparing real devices means setting them from those devices' published figures.
- **The logic model has no timing.** There is no propagation delay, fan-out loading, supply droop or crosstalk, and its soft gate maths is a clean mathematical choice, not a transistor-level transfer curve.
- **The multiply model leaves out wiring and drift.** Wire resistance and sneak currents in crossbars, device drift, temperature, calibration, optical loss and crosstalk are all absent, and inputs arrive as exact digital values.
- **Programming errors are redrawn every shot,** as if each shot used a freshly programmed chip.
- **Only single multiply layers are simulated.** Chains of analog layers, as in a full neural network, are left for later work.

Related work

Each piece of this argument has precedent; what this paper adds is one reproducible model in which logic and multiplying succeed or fail for the same reason.

- **Electronic analog multiplying at scale.** IBM Research's analog AI chip combines 35 million phase-change memory devices across 34 tiles. It matched software accuracy on a small keyword-spotting network and came close on a much larger speech-recognition model ([Ambrogio et al., Nature 620, 768–775, 2023](#)).
- **Optical multiplying at the photon limit.** An optical neural network reached 99% accuracy on handwritten digits at about 3.1 detected photons per multiply, and about 90% at 0.66 photons ([Wang et al., Nature Communications 13, 123, 2022](#)). Its key principle, noise averaging across a dot product's many terms, is the flip side of Result 3: summing many terms helps when the answer only needs to be close, and hurts when it must be exact.
- **Analog probability machines.** Loeliger, Lustenberger, Helfenstein and Tarköy built analog chips that pass probabilities between nodes to decode error-correcting codes ([IEEE Transactions on Information Theory 47\(2\), 837–843, 2001](#)). The team reported all-analog decoders that beat digital ones on speed or power ([Lustenberger et al., ISCAS 1999](#)). One reading in this paper's terms: the code's own redundancy pulls messages back towards valid codewords, so the restoration lives in the maths rather than in each gate.

Appendix A: the arithmetic behind the numbers

Three short formulas reproduce the headline numbers to within a tenth of a bit, or a hundredth of a percent.

Effective bits. R is the converter's full scale (± 5 standard deviations of the answer) and σ is the RMS error. A perfect b -bit converter scores exactly b .

$$b_{\text{eff}} = \log_2 \frac{2R}{\sqrt{12} \sigma}$$

The two currencies. With inputs and weights spread evenly over -1 to $+1$, weight-programming error σ_p and photons per full-scale multiply Φ set these limits:

$$b_{\text{prog}} \approx \log_2 \frac{5}{3 \sigma_p} \quad b_{\text{shot}} \approx \log_2 \frac{10}{3 \sqrt{\Phi}}$$

At $\sigma_p = 2\%$ the first gives 6.38 bits (6.4 measured); at $\Phi = 1,000$ the second gives 5.93 (5.9 measured). Halving σ_p adds one bit, and four times Φ doubles $\sqrt{\Phi}$, which also adds one bit.

Exactness. Summing N values of ± 1 , right answers sit 2 apart, so an answer stays exact while its noise stays under 1. With per-term noise σ_1 :

$$P_{\text{exact}} = \operatorname{erf} \left(\frac{1}{\sqrt{2} \sigma_1 \sqrt{N}} \right)$$

For the crossbar, $\sigma_1 = \sqrt{(0.02^2 + 0.01^2 + 0.005^2)} \approx 0.0229$, the last term being the sense-amplifier noise scaled to one term. At $N = 256$ this gives 99.35%, against 99.34% measured. The converter bits needed grow by half a bit each time N doubles: 7 at 256 terms, 10 at 16,384.

Appendix B: reproduce every number

Every figure here comes from one command with a fixed seed. The simulator needs Rust 1.89 or newer and nothing else.

```
cargo test                # 38 tests, including the laws in Appendix
A
cargo run --release -- depth    # Result 1: whisper test (noise 0.1, 2,000
trials, seed 42)
cargo run --release -- sweep    # Result 1: 8-bit adder sweep (seed 42)
cargo run --release -- trace fulladder 111 --noise 0.12 --gain 0 --seed 3
cargo run --release -- trace fulladder 111 --noise 0.12 --gain 8 --seed 3
cargo run --release -- mac      # Result 2 (200 shots, seed 7)
cargo run --release -- exact    # Result 3 (seed 7)
```

Any knob can be changed on the command line, for example `--program 0.01` or `--photons 10000`, to test a real device's figures.